



第3章 摊还分析

Amortized Analysis

本讲内容

7.1 摊还分析原理

7.2 聚集方法

7.3 会计方法

7.4 势能方法

7.5 动态表操作的摊还分析

基本思想

在摊还分析中，执行一系列数据结构操作所需要时间，是通过对执行的所有操作求平均而得出的

基本思想

对一个数据结构
要执行一系列操作：

- 有的代价很高
- 有的代价一般
- 有的代价很低

将总的代价摊还到
每个操作上

平摊
代价

不涉及概率，
不同于平均情况
分析

本讲内容

7.1 摊还分析原理

7.2 聚集方法

7.3 会计方法

7.4 势能方法

7.5 动态表操作的摊还分析

聚集分析法-原理

操作序列中的每个操作被赋予相同的代价，
不管操作的类型

对数据结构共有 n 个操作
最坏情况下：

操作1: t_1

操作2: t_2

⋮

⋮

⋮

操作 n : t_n

$$T(n) = \sum_{i=1}^n t_i$$

摊还代价:
 $T(n)/n$

聚集分析法实例1-栈操作

普通栈操作：

- $\text{PUSH}(S, x)$ ：将对象 x 压入栈 S ；
- $\text{POP}(S)$ ：弹出并返回 S 的顶端元素，对空栈返回一个错误；

时间代价：

- 两个操作的运行时间都是 $O(1)$
- 我们可把每个操作的代价视为1
- n 个PUSH和POP操作系列的总代价是 n
- n 个操作的实际运行时间为 $\Theta(n)$

聚集分析法实例1-栈操作

新的栈操作，MULTIPOP(S, k):

- 去掉栈 S 的 k 个顶端对象
- 或当 S 中包含少于 k 个对象时弹出整个栈

聚集分析法实例1-栈操作

实际运行时间与实际执行的POP操作数成线性关系

输入： 栈 S , k

输出： 返回 S 顶端 k 个对象

MULTIPOP(S, k):

执行一次While循环要调用一次POP, While循环执行的次数是从栈中弹出的对象数 $\min(s, k)$

1. While not STACK-EMPTY(S) and $k \neq 0$ Do
2. POP(S);
3. $k \leftarrow k - 1$;

MULTIPOP的总代价即为 $\min(s, k)$, 其中 s 是栈 S 中对象个数

聚集分析法实例1-栈操作

- 初始为空的栈上的 n 个栈操作序列的分析
- 由PUSH、POP和MULTIPOP长为 n 的栈操作序列

操作1: t_1

操作2: t_2

:

:

:

操作 n : t_n

最坏情况下，每个操作都是: MULTIPOP，每个MULTIPOP的代价最坏是 n ，因为栈的大小最大是 n 。

$$T(n) = n^2$$

- n 是所有操作的次数，不是输入规模！！
- 这样分析太粗糙了！！它这不是一个确界；
- 我们能不能从元素进出栈的情况来分析？

聚集分析法实例1-栈操作

- 初始为空的栈上的 n 个栈操作序列的分析
- 由PUSH、POP和MULTIPOP长为 n 的栈操作序列
- 一个对象在每次被压入栈后至多被弹出一次
- 在一个非空栈上调用POP的次数（包括在MULTIPOP内的调用）至多等于PUSH的次数，即至多为 $n-1$ （比如 $n-1$ 次PUSH，1次MULTIPOP）

操作1: t_1

操作2: t_2

:

:

:

操作 n : t_n

$$T(n) \leq 2n$$

注意：分析过程没有使用任何的概率！

- 摊还代价 = $T(n)/n = O(1)$;
- 最坏情况下这样的操作序列的时间复杂度最多为 $O(n)$;
- 利用聚集分析，得到了一个更好的上界。

聚集分析法实例2-二进制计数器

1. 问题定义

- 实现一个由 0 开始向上计数的 k 位二进制计数器。
- 输入： k 位二进制变量 x ，初始值为 0。
- 输出： $(x + 1) \bmod 2^k$
- 数据结构：

k 位数组 $A[0 \cdots k - 1]$ 作为计数器，存储 x

x 的最低位在 $A[0]$ 中，最高位在 $A[k - 1]$ 中

$$x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$$

聚集分析法实例2-二进制计数器

2. 计数器加1算法

- 输入： $A[0 \cdots k - 1]$ ，存储二进制数 x
- 输出： $A[0 \cdots k - 1]$ ，存储二进制数 $(x + 1) \bmod 2^k$

INCREMENT(A)

1. $i \leftarrow 0;$
2. while $i < \text{length}[A]$ and $A[i] == 1$ Do
3. $A[i] \leftarrow 0;$
4. $i \leftarrow i + 1;$
5. If $i < \text{length}[A]$ Then
6. $A[i] \leftarrow 1.$

聚集分析法实例2-二进制计数器

3.初始为零的计数器上 n 个INCREMENT操作的分析

Counter									
N	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total
0	0	0	0	0	0	0	0	0	0
1					0	0	0	1	1
2						0	1	0	3
3						0	1	1	4
4						1	0	0	7
5						1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15

该列每8次发生一次改变共 $n/8$ 次

该列每4次发生一次改变共 $n/4$ 次

该列每2次发生一次改变共 $n/2$ 次

该列每次操作发生一次变化共 n 次

总共发生的改变为：
 $\sum \left\lfloor \frac{n}{2^i} \right\rfloor \quad (i = 0, 1, 2, \dots, \log_2 n) < 2n,$
 每个操作的平均代价，
 即摊还代价为 $O(n)/n = O(1)$

- 每次INCREMENT操作的代价与被改变值的字位的个数成线性关系
- 粗略地讲：每次INCREMENT操作最多改变计数器中 k 位， n 次INCREMENT操作，代价为 nk

本讲内容

7.1 摊还分析原理

7.2 聚集方法

7.3 会计方法

7.4 势能方法

7.5 动态表操作的摊还分析

会计方法-基本原理

- 一个操作序列中有不同类型的操作，不同类型的操作代价各不相同
 - 为每种操作分配不同的摊还代价
 - 摊还代价可能比实际代价大，也可能比实际代价小
- 操作被执行时，支付摊还代价
 - 如果摊还代价比实际代价高：摊还代价的一部分用于支付实际代价，多余部分作为存款附加在数据结构的**具体数据对象**上
 - 如果摊还代价比实际代价低：摊还代价及**数据对象上的存款**用来支付实际代价
- 摊还代价的总和与实际代价的总和的关系
 - 只要我们能保证：在任何操作序列上，**存款的总额非负**，则所有操作摊还代价的总和就是实际代价总和的上界（**摊还代价大于等于实际代价**）

会计方法-基本原理

于是：我们在各种操作上定义平摊代价使得任意操作序列上存款总量是**非负的**，将操作序列上平摊代价求和即可得到这个操作序列的复杂度上界

会计方法实例 1—栈操作

1. 各栈操作的实际代价:

PUSH 1,

POP 1,

MULTIPOP $\min(s, k)$

2. 各栈操作的摊还代价:

PUSH 2,

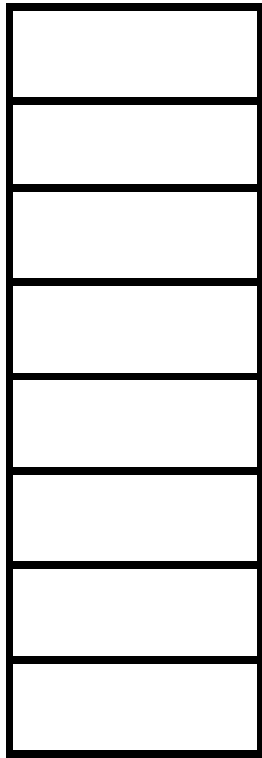
POP 0,

MULTIPOP 0,

能不能保证在任何操作序列上, 存款的总额非负?

会计方法实例 1—栈操作

• 栈操作序列代价分析



弹出一个元素支付摊还代价0，附着在数据对象上的存款1与摊还代价0一起支付实际代价1

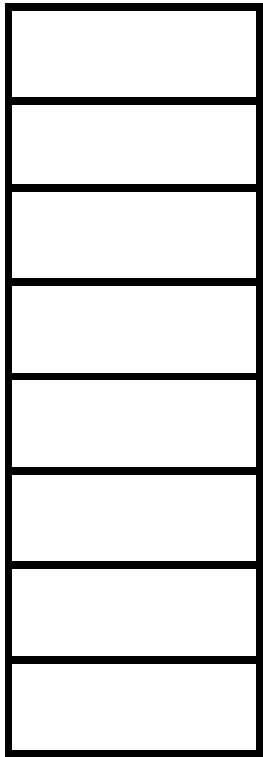
插入一个元素支付摊还代价2，1用于支付实际代价，1作为存款附着在数据对象上

操作被执行时，支付摊还代价

- 如果摊还代价比实际代价高：摊还代价的一部分用于支付实际代价，多余部分作为存款附加在数据结构的具体数据对象上
- 如果摊还代价比实际代价低：摊还代价及数据对象上的存款用来支付实际代价

会计方法实例 1—栈操作

- 栈操作序列代价分析



- 只要我们的操作序列是合理的，则可以保证存款总和
负
- 于是所有操作的摊还代价总和就是操作序列实际代价总
和的上界=?

长度为 n 的操作序列中：PUSH操作的个数 $\leq n$
于是：摊还代价的总和 $\leq 2n$ ，所以操作序列
的实际代价为 $O(n)$

会计方法实例 2—二进制计数器

- 计数器加1算法

- 输入： $A[0 \cdots k - 1]$ ，存储二进制数 x
- 输出： $A[0 \cdots k - 1]$ ，存储二进制数 $(x + 1) \bmod 2^k$

INCREMENT(A)

```
1.    $i \leftarrow 0$ ;  
2.   while  $i < \text{length}[A]$  and  $A[i] == 1$  Do  
3.        $A[i] \leftarrow 0$ ;  
4.        $i \leftarrow i + 1$ ;  
5.   If  $i < \text{length}[A]$  Then  
6.        $A[i] \leftarrow 1$ .
```

会计方法实例 2—二进制计数器

- 初始为零的计数器上 n 个INCREMENT操作的分析

显然：这个操作序列的代价与0-1或者1-0翻发生的次数成正比

定义：

0-1 翻转的摊还代价为2

1-0 翻转的摊还代价为0

会计方法实例 2—二进制计数器

- 初始为零的计数器上 n 个INCREMENT操作的分析

定义：

0-1翻转的摊还代价为2

1-0翻转的摊还代价为0

任何操作序列，存款余额是计数器中1的个数，非负
因此，所有的翻转操作的摊还代价的和是这个操作
序列代价的上界

会计方法实例 2—二进制计数器

- 初始为零的计数器上 n 个INCREMENT操作的分析

定义：

0-1翻转的摊还代价为2

1-0翻转的摊还代价为0

对每个INCREMENT操作：

- 找到右起的第一个0，将他翻转成1——支付摊还代价2
- 将这个0之后的所有1翻转成0——支付摊还代价0
- 对这个INCREMENT操作而言，支付了摊还代价2

0-1支付摊还代价
2, 1用于支付实
际代价, 1作为存
款附着在数据对
象上

1-0支付摊还代价
0, 附着在数据对
象上的存款1与摊
还代价0一起支付
实际代价1

0-1支付摊还代价
2, 1用于支付实
际代价, 1作为存
款附着在数据对
象上

Counter											total
N	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]			
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	1+1	1	1
2	0	0	0	0	0	0	1+1	0	0	3	3
3	0	0	0	0	0	0	1+1	1+1	1+1	4	4
4	0	0	0	0	0	1+1	0	0	0	7	7
5	0	0	0	0	0	1+1	0	0	1+1	8	8
6	0	0	0	0	0	1+1	1+1	0	0	10	10
7	0	0	0	0	0	1+1	1+1	1+1	1+1	11	11
8	0	0	0	0	1+1	0	0	0	0	15	15

- 任何操作序列, 存款余额是计数器中1的个数, 非负。为什么?
- 因此, 所有的翻转操作的摊还代价的和是这个操作序列代价的上界。

会计方法实例 2—二进制计数器

- 初始为零的计数器上 n 个INCREMENT操作的分析

定义：

0-1翻转的摊还代价为2

1-0翻转的摊还代价为0

对于长度为 n 的INCREMENT操作序列：

- 支付的摊还代价的总和为 $2n$
- 因此，这样一个操作序列的复杂度上界为 $2n$

本讲内容

7.1 摊还分析原理

7.2 聚集方法

7.3 会计方法

7.4 势能方法

7.5 动态表操作的摊还分析

势能分析—基本原理

- 在会计方法中，如果操作的摊还代价比实际代价大，我们将余额与具体的**数据对象**关联
- 如果我们将这些余额都与**整个数据结构**关联，所有的这样的余额之和，构成——数据结构的**势能**
- 如果操作的摊还代价大于操作的实际代价-**势能增加**
- 如果操作的摊还代价小于操作的实际代价，要用数据结构的势能来支付实际代价-**势能减少**

势能分析—基本原理

势能的定义：对一个初始数据结构 D_0 执行 n 个操作，
对操作 i ：

- 实际代价 c_i 将数据结构 D_{i-1} 变为 D_i
- 势能函数 ϕ 将每个数据结构 D_i 映射为一个实数 $\phi(D_i)$
- $\phi(D_i)$ 就是关联到数据结构 D_i 的势能
- 摊还代价 c'_i 定义为： $c'_i = c_i + \phi(D_i) - \phi(D_{i-1})$
- 势能函数 ϕ 是非递减的

势能分析—基本原理

- n 个操作的总的摊还代价为：

$$\begin{aligned}\sum_{i=1}^n c'_i &= \sum_{i=1}^n (c_i + \phi(D_i) - \phi(D_{i-1})) \\ &= \sum_{i=1}^n c_i + (\phi(D_n) - \phi(D_0))\end{aligned}$$

摊还代价依赖于所选择的势函数 ϕ 。不同的势函数可能会产生不同的摊还代价，但它们都是实际代价的上界

在实践中，我们定义 $\phi(D_0)$ 为0，然后再证明对所有 i 有 $\phi(D_i) \geq 0$

于是势能函数 ϕ 满足 $\phi(D_n) \geq \phi(D_0)$ ，则总的摊还代价就是总的实际代价的一个上界

势能方法实例1——栈操作

- $\phi(D)$ = 栈 D 中对象的个数
- 初始空栈 D_0 , $\phi(D_0) = 0$
- 因为栈中的对象数始终非负, 第 i 个操作之后的栈 D_i 满足 $\phi(D_i) \geq 0 = \phi(D_0)$
- 于是: 以势能函数 $\phi(D)$ 表示的 n 个操作的摊还代价的总和就表示了实际代价的一个上界

势能方法实例1——栈操作

- 作用于包含 s 个对象的栈上的栈操作的摊还代价

如果第 i 个操作是PUSH操作

- 实际代价： $c_i = 1$
- 势差： $\phi(D_i) - \phi(D_{i-1}) = (s + 1) - s = 1$
- 摊还代价： $c'_i = c_i + \phi(D_i) - \phi(D_{i-1}) = 1 + 1 = 2$

势能方法实例1——栈操作

- 作用于包含 s 个对象的栈上的栈操作的摊还代价

如果第 i 个操作是POP

- 实际代价： $c_i = 1$
- 势差： $\phi(D_i) - \phi(D_{i-1}) = -1$
- 摊还代价： $c'_i = c_i + \phi(D_i) - \phi(D_{i-1}) = 1 - 1 = 0$

势能方法实例1——栈操作

- 作用于包含 s 个对象的栈上的栈操作的摊还代价

如果第 i 个操作是MULTIPOP(S, k)且弹出了 $k' = \min(s, k)$ 个对象

- 实际代价: $c_i = k'$
- 势差: $\phi(D_i) - \phi(D_{i-1}) = -k'$
- 摊还代价: $c'_i = c_i + \phi(D_i) - \phi(D_{i-1}) = k' - k' = 0$

这也间接地给出了会计法为什么设置

PUSH 2,

POP 0,

MULTIPOP 0,

势能方法实例1——栈操作

- 作用于包含 s 个对象的栈上的栈操作的摊还代价

摊还分析：

- 每个栈操作的摊还代价都是 $O(1)$
- n 个操作序列的总摊还代价就是 $O(n)$
- 因为 $\phi(D_i) \geq \phi(D_0)$ ， n 个操作的总摊还代价即为总的实际代价的一个上界，即 n 个操作的最坏情况代价为 $O(n)$

势能方法实例 2 —— 二进制计数器

- $\phi(D)$ = 计数器 D 中 1 的个数
- 计数器初始状态 D_0 中 1 的个数为 0, $\phi(D_0) = 0$
- 因为数组中的 1 的个数始终为非负, 第 i 个操作之后的 D_i 满足 $\phi(D_i) \geq 0 = \phi(D_0)$
- 于是: n 个操作的摊还代价的总和就表示了实际代价的一个上界

势能方法实例 2 —— 二进制计数器

- 第 i 次INCREMENT操作的摊还代价

设第 i 次INCREMENT操作对 t_i 个位进行了 置0，至多（一定）将一位 置1

- 该操作的实际代价： $c_i = t_i + 1$
- 在第 i 次操作后计数器中1的个数为 $b_i \leq b_{i-1} - t_i + 1$
- 势差： $\phi(D_i) - \phi(D_{i-1}) \leq (b_{i-1} - t_i + 1) - b_{i-1} = 1 - t_i$
- 摊还代价： $c'_i = c_i + \phi(D_i) - \phi(D_{i-1}) \leq (t_i + 1) + (1 - t_i) = 2$

势能方法实例 2 — 二进制计数器

- 第 i 次INCREMENT操作的摊还代价

计数器初始状态为0时的摊还分析：

- 每个操作的摊还代价都是 $O(1)$
- n 个操作序列的总摊还代价就是 $O(n)$
- 因为 $\phi(D_i) \geq \phi(D_0)$ ， n 个操作的总摊还代价即为总的实际代价的一个上界，即 n 个操作的最坏情况代价为 $O(n)$

势能方法实例 2 —— 二进制计数器

- 开始时**不为零**的计数器上 n 个INCREMENT操作的分析

- 设开始时有 b_0 个1且 $b_0 \geq 0$
- 在 n 次INCREMENT操作之后有 b_n 个1
- 一系列操作的实际代价为：

$$\sum_{i=1}^n c_i = \sum_{i=1}^n c'_i - \phi(D_n) + \phi(D_0)$$

- 因为 $\phi(D_0) = b_0$, $\phi(D_n) = b_n$, n 次INCREMENT操作的总实际代价为：

$$\sum_{i=1}^n c_i = \sum_{i=1}^n 2 - b_n + b_0 = 2n - b_n + b_0$$

- 如果我们执行了至少 $n = \Omega(k)$ 次INCREMENT操作，则无论计数器中包含什么样的初始值，总的实际代价都是 $O(n)$ 。

本讲内容

7.1 摊还分析原理

7.2 聚集方法

7.3 会计方法

7.4 势能方法

7.5 动态表操作的摊还分析

动态表

- 本节的目的：
 - 研究表的动态扩张和收缩的问题
 - 利用摊还分析证明插入和删除操作的摊还代价为 $O(1)$ ，即使当它们引起了表的扩张和收缩时具有较大的实际代价（表的扩张和收缩不是一直发生的）
 - 研究如何保证一动态表中未用的空间始终不超过整个空间的一部分

动态表—基本术语

- 动态表支持的操作
 - **TABLE-INSERT**: 将某一元素插入表中
 - **TABLE-DELETE**: 将一个元素从表中删除
- 数据结构: 用一个（一组）数组来实现动态表
- 非空表 T 的装载因子 $\alpha(T) = \frac{T \text{ 存储的对象数}}{\text{表大小}}$
 - 空表的大小为0，装载因子为1
 - 如果动态表的装载因子以一个常数为下界，则表中未使用的空间就始终不会超过整个空间的一个常数部分

动态表——基本术语

设 T 表示一个表：

- $\text{table}[T]$ 是一个指向非空表的存储块的指针
- $\text{num}[T]$ 包含了表中的项数
- $\text{size}[T]$ 是 T 的大小
- 装载因子 $\alpha(T) = \frac{T \text{ 存储的对象数}}{\text{表大小}} = \frac{\text{num}[T]}{\text{size}[T]}$
- 开始时, $\text{num}[T] = \text{size}[T] = 0$, $\alpha(T) = 1$

动态表——表的扩张

- 向表中插入一个数组元素时，分配一个包含比原表更多的槽的新表，再将原表中的各项复制到新表中去
- 一种常用的启发式技术是分配一个比原表大一倍的新表，如果只对表执行插入操作，则表的装载因子总是至少为 $\frac{1}{2}$ ，这样浪费掉的空间就始终不会超过表总空间的一半

动态表——表的扩张

算法: TABLE-INSERT(T, x)

1. If $size[T] == 0$ Then /*空表*/
2. allocate $table[T]$ with 1 slot (槽) ;
3. $size[T] \leftarrow 1$;
4. If $num[T] == size[T]$ Then /*满表*/
5. allocate new table with $2 \times size[T]$ slots;
6. insert all items in $table[T]$ into new-table;
7. free $table[T]$;
8. $table[T] \leftarrow$ new-table;
9. $size[T] \leftarrow 2 \times size[T]$;
10. Insert x into $table[T]$;
11. $num[T] \leftarrow num[T] + 1$.

动态表—表的扩张

初始为空的表上 n 次TABLE-INSERT操作的代价分析-粗略分析

算法: TABLE-INSERT(T, x)

1. If $size[T] == 0$ Then
2. allocate $table[T]$ with 1 slot;
3. $size[T] \leftarrow 1$;
4. If $num[T] == size[T]$
5. allocate new table with $2 \times size[T]$ slots;
6. insert all items in $table[T]$ into new-table;
7. free $table[T]$;
8. $table[T] \leftarrow$ new-table;
9. $size[T] \leftarrow 2 \times size[T]$;
10. Insert x into $table[T]$;
11. $num[T] \leftarrow num[T] + 1$.

第 i 次操作的代价 C_i :

如果 $i = 1$ $C_i = 1$

如果表有空间 $C_i = 1$

如果表是满的 $C_i = i$

如果以共有 n 次操作, 最坏情况下:

每次进行 n 次操作, 总的代价上界为 n^2

这个界不精确, 因为执行 n 次TABLE-INSERT操作的过程中并不常常包括扩张表的代价。仅当 $i-1$ 为2的整数幂时第 i 次操作才会引起一次表的扩张。

动态表—表的扩张

初始为空的表上 n 次TABLE-INSERT操作的代价分析-聚集分析

算法: TABLE-INSERT(T, x)

1. If $size[T] == 0$ Then
2. allocate $table[T]$ with 1 slot;
3. $size[T] \leftarrow 1$;
4. If $num[T] == size[T]$ Then
5. allocate new table with n
6. insert all items in $table[T]$
7. free $table[T]$;
8. $table[T] \leftarrow$ new-table;
9. $size[T] \leftarrow 2 \times size[T]$;
10. Insert x into $table[T]$;
11. $num[T] \leftarrow num[T] + 1$.

第 i 次操作的代价 c_i :

如果 $i = 2^j + 1$ $c_i = i$

否则 $c_i = 1$

n 次TABLE-INSERT操作的总代价为:

$$\sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lceil \lg n \rceil} 2^j < n + 2n = 3n$$

每一操作的摊还代价为 $3n/n = 3$

动态表——表的扩张

初始为空的表上 n 次TABLE-INSERT操作的代价分析-会计法分析

算法: TABLE-INSERT(x)

任何时候, 存款总和 ≥ 0

1. If $size[T] == 0$ Then
2. allocate $table[T]$ with $size[T]$ slots;
3. $size[T] \leftarrow 1$;
4. If $num[T] == size[T]$ Then
5. allocate new table with $2 \times size[T]$ slots;
6. insert all items in $table[T]$ into new-table;
7. free $table[T]$;
8. $table[T] \leftarrow$ new-table;
9. $size[T] \leftarrow 2 \times size[T]$;
10. Insert x into $table[T]$;
11. $num[T] \leftarrow num[T] + 1$.

每次执行TABLE-INSERT摊还代价为3

1支付第10步中的基本插入操作的实际代价

1作为自身的存款

1存入表中第一个没有存款的数据上

当发生表的扩张时, 数据的复制的代价由数据上的存款来支付

初始为空的表上 n 次TABLE-INSERT操作的摊还代价总和为 $3n$

为什么每次执行TABLE-INSERT摊还代价为3?

《算法导论》P266

动态表—表的扩张

初始为空的表上 n 次TABLE-INSERT操作的代价分析-会计法分析

	1
存款	1

插入1（注：第一次摊还代价为2，其余为3）

	1	
存款	0	

扩张

	1	2
存款	1	1

插入2

	1	2		
存款	0	0		

扩张

	1	2	3	
存款	1	0	1	

插入3

每次执行TABLE—INSERT摊还代价为3:

- 1支付第10步中的基本插入操作的实际代价
- 1作为自身的存款
- 1存入表中第一个没有存款的数据上
- 当发生表的扩张时，数据的复制的代价由数据上的存款来支付
- 初始为空的表上 n 次TABLE-INSERT操作的摊还代价总和为 $3n$

动态表——表的扩张

初始为空的表上 n 次TABLE-INSERT操作的代价分析-会计法分析

i	1	2	3	4
存款	1	1	1	1

插入4

i	1	2	3	4				
存款	0	0	0	0				

扩张

i	1	2	3	4	5			
存款	1	0	0	0	1			

插入5

i	1	2	3	4	5	6		
存款	1	1	0	0	1	1		

插入6

依此类推

动态表——表的扩张

初始为空的表上 n 次TABLE-INSERT操作的代价分析-势能法分析

- 势能函数怎么定义，才能使得表满发生扩张时势能能支付扩张的代价
- 如果势能函数满足：
 - 刚扩充完（还未插入）， $\phi(T) = 0$ ；
 - 表满时 $\phi(T) = size(T)$ ，是否可行？

势能函数可行并且：

- $num[T] \geq size[T]/2$, $\phi(T) \geq 0$
- n 次TABLE-INSERT操作的总的摊还代价就是总的实际代价的一个上界

算法：TABLE-INSERT(T, x)

1. If $size[T] == 0$ Then
2. allocate $table[T]$ with 1 slot;
3. $size[T] \leftarrow 1$;
4. If $num[T] == size[T]$ Then
5. allocate new table with $2 \times size[T]$ slots;
6. insert all items in $table[T]$ into new-table;
7. free $table[T]$;
8. $table[T] \leftarrow$ new-table;
9. $size[T] \leftarrow 2 \times size[T]$;
10. Insert x into $table[T]$;
11. $num[T] \leftarrow num[T] + 1$.

动态表——表的扩张

初始为空的表上 n 次TABLE-INSERT操作的代价分析-势能法分析

算法: TABLE-INSERT(T, x)

1. If $size[T] == 0$ Then
2. allocate $table[T]$ with 1 slot;
3. $size[T] \leftarrow 1$;
4. If $num[T] == size[T]$ Then
5. allocate new table with $2 \times size[T]$ slots;
6. insert all items in $table[T]$ into new-table;
7. free $table[T]$;
8. $table[T] \leftarrow$ new-table;
9. $size[T] \leftarrow 2 \times size[T]$;
10. Insert x into $table[T]$;
11. $num[T] \leftarrow num[T] + 1$.

$$\phi(T) = 2 * num[T] - size[T]$$

第 i 次操作的摊还代价:

$$c'_i = c_i + \phi(T_i) - \phi(T_{i-1})$$

如果发生扩张: $c'_i = 3$

发生扩张: $c_i = num_i$

$$size_i = 2 * size_{i-1}; num_{i-1} = size_{i-1} = num_i - 1 \Rightarrow size_i = 2 * (num_i - 1)$$

势能

$$\begin{aligned} c'_i &= c_i + \phi(T_i) - \phi(T_{i-1}) = num_i + (2 * num_i - size_i) - (2 * num_{i-1} - size_{i-1}) \\ &= num_i + (2 * num_i - 2 * (num_i - 1)) - (2(num_i - 1) - (num_i - 1)) \\ &= num_i + 2 - num_i + 1 \\ &= 3 \end{aligned}$$

动态表——表的扩张

初始为空的表上 n 次TABLE-INSERT操作的代价分析-势能法分析

算法: TABLE-INSERT(T, x)

1. If $size[T] == 0$ Then
2. allocate $table[T]$ with 1 slot;
3. $size[T] \leftarrow 1$;
4. If $num[T] == size[T]$ Then
5. allocate new table with $2 \times size[T]$ slots;
6. insert all items in $table[T]$ into new-table;
7. free $table[T]$;
8. $table[T] \leftarrow$ new-table;
9. $size[T] \leftarrow 2 \times size[T]$;
10. Insert x into $table[T]$;
11. $num[T] \leftarrow num[T] + 1$.

$$\phi(T) = 2 * num[T] - size[T]$$

第 i 次操作的摊还代价:

$$c'_i = c_i + \phi(T_i) - \phi(T_{i-1})$$

如果发生扩张: $c'_i = 3$

否则 $c'_i = 3$

初始为空的表上 n 次TABLE-INSERT操作的摊还代价总和为 $3n$

不扩张: $c_i = 1$

$$size_i = size_{i-1}; num_{i-1} = num_i - 1$$

势能

$$\begin{aligned} c'_i &= c_i + \phi(T_i) - \phi(T_{i-1}) = 1 + (2 * num_i - size_i) - (2 * num_{i-1} - size_{i-1}) \\ &= 1 + (2 * num_i - size_i) - (2(num_i - 1) - size_i) \\ &= 1 + 2 \\ &= 3 \end{aligned}$$

动态表

- 动态表支持的操作
 - TABLE-INSERT: 将某一元素插入表中
 - TABLE-DELETE: 将一个元素从表中删除
- 非空表 T 的装载因子 $\alpha(T) = \frac{T \text{ 存储的对象数}}{\text{表大小}}$
 - 空表的大小为0, 装载因子为1
 - 如果动态表的装载因子以一个常数下界, 则表中未使用的空间就始终不会超过整个空间的一个常数部分
- 理想情况下, 我们希望动态表满足:
 - 表具有一定的丰满度
 - 表的操作序列的复杂度是线性的

动态表-表的扩张

- 向表中插入一个数组元素时，分配一个包含比原表更多的槽的新表，再将原表中的各项复制到新表中去
- 分配一个比原表大一倍的新表，如果只对表执行插入操作，则表的装载因子总是至少为 $1/2$ ，这样浪费掉的空间就始终不会超过表总空间的一半

动态表-表的收缩（自学内容）

根据表的扩张策略，很自然地想到表的收缩策略：

- 当表的装载因子小于 $1/2$ 时，收缩表为原表的一半

是否合适？



动态表-表的扩张

n 是2的方幂，下面的一个长度为 n 的操作序列：

- 前 $n/2$ 个操作是插入，
- 之后跟IDDIIIDDII…，I表示插入操作，D表示删除操作。

每次扩张和收缩的代价为 $O(n)$ ，共有 $O(n)$ 次扩张或收缩
总代价为 $O(n^2)$ ，而每一次操作的摊还代价为 $O(n)$ ，每个操作
的摊还代价太高！！

动态表-表的扩张和收缩

改进表的收缩策略：

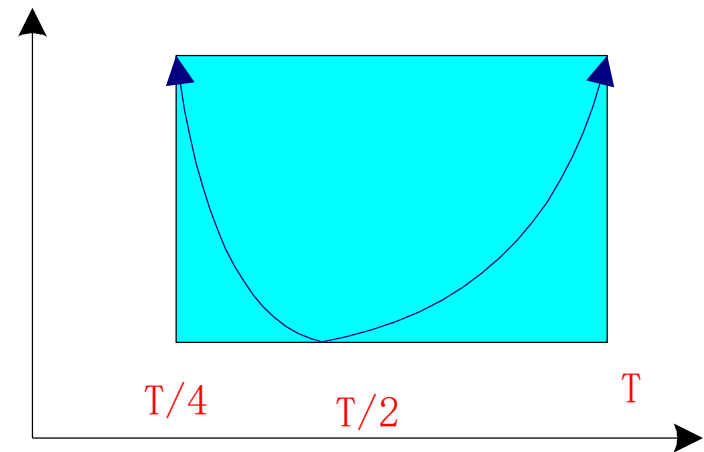
- 当删除元素时，允许装载因子低于 $1/2$
- 但当删除一项而引起表的装载因子不足 $1/4$ 满时，我们就将表缩小为原来的一半
- 当向满的表中插入一项时，还是将表扩大一倍
- 这样扩张和收缩过程都使得表的装载因子变为 $1/2$ ，但是表的装载因子的下界是 $1/4$

动态表-表的扩张和收缩

- 由 n 个TABLE-INSERT和TABLE-DELETE操作构成序列的代价分析-势能法
- 势能函数
 - 操作序列过程中表 T 的势总是非负的；这样才能保证一系列操作的总摊还代价即为其实际代价的一个上界
 - 表的扩张和收缩过程要消耗大量的势

势能的要求：

- 1、 $num(T) = size(T)/2$ 时，势最小
- 2、当 $num(T)$ 减小时，势增加直到收缩
- 3、当 $num(T)$ 增加时，势增加直到扩张



动态表-表的扩张和收缩

- 由 n 个TABLE-INSERT和TABLE-DELETE操作构成序列的代价分析-势能法
- 势函数定义

$$\phi(T) = \begin{cases} 2 \cdot \text{num}[T] - \text{size}[T] & \alpha(T) \geq 1/2 \\ \text{size}[T]/2 - \text{num}[T] & \alpha(T) < 1/2 \end{cases}$$

空表的势为0，且势总是非负的。这样 ϕ 表示的一系列操作的总摊还代价即为其实际代价的一个上界

动态表-表的扩张和收缩

- 由n个TABLE-INSERT和TABLE-DELETE操作构成序列的代价分析-势能法
- 势函数的某些性质：
$$\phi(T) = \begin{cases} 2 \cdot \text{num}[T] - \text{size}[T] & \alpha(T) \geq 1/2 \\ \text{size}[T]/2 - \text{num}[T] & \alpha(T) < 1/2 \end{cases}$$
 - 当装载因子为1/2时，势为0。
 - 当装载因子为1时，有 $\text{num}[T] = \text{size}[T]$ ，这就意味着 $\phi(T) = \text{num}[T]$ 。
这样当因插入一项而引起一次扩张时，就可用势来支付其代价。
 - 当装载因子为1/4时， $\text{size}[T] = 4 * \text{num}[T]$ ，它意味着 $\phi(T) = \text{num}[T]$ 。
因而当删除某项引起一次收缩时就可利用势来支付其代价。

动态表-表的扩张和收缩

- 由 n 个TABLE-INSERT和TABLE-DELETE操作构成序列的代价分析-势能法

- 势函数定义

$$\phi(T) = \begin{cases} 2 \cdot \text{num}[T] - \text{size}[T] & \alpha(T) \geq 1/2 \\ \text{size}[T]/2 - \text{num}[T] & \alpha(T) < 1/2 \end{cases}$$

第 i 次操作的摊还代价: $c'_i = c_i + \phi(T_i) - \phi(T_{i-1})$

第 i 次操作是TABLE-INSERT: 未扩张 $c'_i \leq 3$

第 i 次操作是TABLE-INSERT: 扩张 $c'_i \leq 3$

第 i 次操作是TABLE-DELETE: 未收缩 $c'_i \leq 3$

第 i 次操作是TABLE-DELETE: 收缩 $c'_i \leq 3$

所以作用于一动态表上的 n 个操作的实际时间为 $O(n)$

第*i*次操作是TABLE-INSERT:

➤ $\alpha_{i-1} \geq 1/2$, 则 $\alpha_i > 1/2$, 进一步分为两类:

➤ $\alpha_i = 1$, 即发生扩张; PPT 52页

➤ $1/2 < \alpha_i < 1$, 即未发生扩张; PPT 53页

➤ $\alpha_{i-1} < 1/2$, 则第*i*个操作一定不会产生扩张, 但仍需要区分 α_i

➤ $\alpha_i < 1/2$, $c'_i = ci + \phi(D_i) - \phi(D_{i-1})$

$$= 1 + \left(\frac{size_i}{2} - num_i\right) - \left(\frac{size_{i-1}}{2} - num_{i-1}\right)$$

$$= 1 + \left(\frac{size_i}{2} - num_i\right) - \left(\frac{size_i}{2} - (num_i - 1)\right) = 0$$

➤ $\alpha_i \geq 1/2$, $c'_i = ci + \phi(D_i) - \phi(D_{i-1})$

$$= 1 + (2num_i - size_i) - \left(\frac{size_{i-1}}{2} - num_{i-1}\right)$$

$$= 1 + (2(num_{i-1} + 1) - size_{i-1}) - \left(\frac{size_{i-1}}{2} - num_{i-1}\right)$$

$$= 3 + 3num_{i-1} - \frac{3size_{i-1}}{2} < 3 + \frac{3size_{i-1}}{2} - \frac{3size_{i-1}}{2} = 3$$

第*i*次操作是TABLE-DELETE:

➤ $\alpha_{i-1} < 1/2$, 则必须考虑是否因其表收缩。进一步分为两类:

➤ $\alpha_i < 1/4$ (发生表收缩, 即 $\alpha_{i-1} = \frac{\text{num}_{i-1}}{\text{size}_{i-1}} = 1/4$),

$$\begin{aligned} c'_i &= ci + \phi(D_i) - \phi(D_{i-1}) \\ &= (\text{num}_{i-1} + 1) + \left(\frac{\text{size}_i}{2} - \text{num}_i \right) - \left(\frac{\text{size}_{i-1}}{2} - \text{num}_{i-1} \right) \\ &= (\text{num}_{i-1} + 1) + \left(\frac{\text{size}_{i-1}}{4} - (\text{num}_{i-1} - 1) \right) \\ &\quad - \left(\frac{\text{size}_{i-1}}{2} - \text{num}_{i-1} \right) \qquad \begin{array}{l} \text{size}_{i-1} = 4\text{num}_{i-1} \\ \text{size}_i = \text{size}_{i-1}/2 \end{array} \\ &= (\text{num}_{i-1} + 1) + \left(\frac{4\text{num}_{i-1}}{4} - (\text{num}_{i-1} - 1) \right) \\ &\quad - \left(\frac{4\text{num}_{i-1}}{2} - \text{num}_{i-1} \right) \\ &= 2 \end{aligned}$$

第*i*次操作是TABLE-DELETE:

➤ $\alpha_{i-1} < 1/2$ ，则必须考虑是否因其表收缩。进一步分为两类：

➤ $1/4 \leq \alpha_i < 1/2$ （未发生表收缩），

$$\begin{aligned} c'_i &= ci + \phi(D_i) - \phi(D_{i-1}) \\ &= 1 + \left(\frac{size_i}{2} - num_i \right) - \left(\frac{size_{i-1}}{2} - num_{i-1} \right) \\ &= 1 + \left(\frac{size_{i-1}}{2} - (num_{i-1} - 1) \right) - \left(\frac{size_{i-1}}{2} - num_{i-1} \right) \\ &= 2 \end{aligned}$$

第*i*次操作是TABLE-DELETE:

➤ $\alpha_{i-1} \geq 1/2$, 则第*i*个操作一定不会产生收缩, 但仍需要区分 α_i

➤ $\alpha_i < 1/2$, $c'_i = ci + \phi(D_i) - \phi(D_{i-1})$

$$= 1 + \left(\frac{size_i}{2} - num_i\right) - (2num_{i-1} - size_{i-1})$$

$$= 1 + \left(\frac{size_{i-1}}{2} - (num_{i-1} - 1)\right) - (2num_{i-1} - size_{i-1})$$

$$= 2 + \frac{3size_{i-1}}{2} - 3num_{i-1} < 2 \quad \alpha_{i-1} = \frac{num_{i-1}}{size_{i-1}} \geq 1/2$$

➤ $\alpha_i \geq 1/2$, $c'_i = ci + \phi(D_i) - \phi(D_{i-1})$

$$= 1 + (2num_i - size_i) - (2num_{i-1} - size_{i-1})$$

$$= 1 + (2(num_{i-1} - 1) - size_{i-1}) - (2num_{i-1} - size_{i-1}) \\ = -1$$

所以作用于一动态表上的*n*个操作的实际时间为 $O(n)$